Context-aware vulnerability classification with knowledge graphs and security intelligence

Krit Meesiri, Panod Manodee, Apichat Laohawan, Sujitta Sonna
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6622780037@g.siit.tu.ac.th, 6622782033@g.siit.tu.ac.th, 6622770491@g.siit.tu.ac.th, 6622770640@g.siit.tu.ac.th

***Abstract*—Modern Security Operations Centers (SOCs) rely on heterogeneous monitoring platforms such as Wazuh, Splunk, Microsoft Sentinel, and AMiner, whose outputs differ in format and abstraction level, requiring analysts to manually correlate security findings across tools. Moreover, standardized vulnerability representations such as CVE, CWE, and CVSS describe intrinsic vulnerability characteristics but provide limited information about the operational context in which vulnerabilities are deployed. This paper presents a context-aware vulnerability classification and risk prioritization system that addresses these challenges through a unified security analysis pipeline. Raw security logs are processed using a DeBERTa-BiLSTM-CRF model to extract security entities and transform heterogeneous findings into a unified structured representation. The same DeBERTa encoder is leveraged by an embedding-based Multi-Layer Perceptron (MLP) classifier, which fuses semantic, numerical, and categorical features to assign an initial risk classification to each event. The extracted entities and initial risk classifications are subsequently integrated into a Vulnerability Context Knowledge Graph (VCKG), implemented using Neo4j, which models relationships among assets, vulnerabilities, software, services, and security events. By incorporating environmental context from the VCKG, the system performs context-aware risk prioritization and categorizes risks into actionable levels—Critical, High, Medium, and Low—and generates detailed threat reports with recommended mitigation measures. The proposed approach enables SOC analysts to move beyond fragmented, tool-specific findings toward a unified and context-driven understanding of organizational security risk.**

I. Introduction

Modern Security Operations Centers (SOCs) rely on heterogeneous monitoring platforms such as Wazuh, Splunk, Microsoft Sentinel, and AMiner, each producing findings in different formats and levels of abstraction. This forces analysts to manually correlate information across tools to understand a single incident. Furthermore, standardized vulnerability representations such as CVE, CWE, and CVSS describe a vulnerability's intrinsic characteristics but do not reflect the environment in which it is deployed—factors such as asset criticality, network exposure, and observed threat activity. As a result, the same vulnerability may pose substantially different levels of risk depending on its deployment context, motivating the need for a context-aware approach to vulnerability classification and prioritization.

Prior work has addressed parts of this problem in isolation. Knowledge graph-based approaches [1], [4], [8], [9], [10] model relationships among vulnerabilities, assets, and threat intelligence, but typically rely on manually curated or report-derived intelligence rather than heterogeneous operational security logs and lack a dedicated learning-based extraction stage. Classification-based approaches such as V2W-BERT [6] and DeKeDVer [2] map vulnerability descriptions to categories such as CWE, but operate on the description in isolation, producing context-free classifications that do not reflect the deployment environment. Consequently, existing approaches do not provide a unified pipeline that combines entity-level extraction from heterogeneous security logs, machine learning-based classification, and graph-based context modeling to produce an environment-aware risk assessment rather than a static vulnerability label.

This paper addresses this gap by proposing a context-aware pipeline for vulnerability classification and prioritization. External threat intelligence and local computing asset data are collected and processed to construct two knowledge graphs (KGs): a Cyber Threat Intelligence (CTI) knowledge graph and a Computer Assets (CA) knowledge graph. These are then integrated into a unified Vulnerability Knowledge Graph (VKG) that captures the relationships between known threats and the organization's assets. Security alerts generated by monitoring tools such as Wazuh, Suricata, and Microsoft Sentinel are used to query the VKG and identify the affected assets. The base scores of the reported vulnerabilities are then recalculated using weights derived from the affected assets, yielding an Environmental Vulnerability Score (EVS) that reflects the organization's specific operational context. The main contributions of this paper is as follows:

- **A Domain-specific Threat Intelligence-assisted Knowledge Graph Representation** We implement a knowledge graph that represents the relationships and correlations between a domain-specific computer assets knowledge graph and a cybersecurity threat intelligence.
- **Context-Aware Classification Model** We design a model to compute the environmental vulnerability score and classify the vulnerability according to the environmental context of the system.
- **Cybersecurity tool integration** We integrate commercial cybersecurity tools in our design as the trigger to query the VKG to calculate the EVS
- **Context-Aware Threat Report** We LLM to interpret the output of our model to compile a threat report of the detected vulnerability from the cybersecurity tool with the vulnerability, affected assets, base CVSS score, and our EVS score.

II. Related Work

Nagaty et al. [1] developed a knowledge graph unification pipeline that constructs and integrates threat intelligence and computer asset knowledge graphs. Their work proposes a five-phase framework:

1) **Data collection.** Two data streams are gathered: threat data from public cyber threat intelligence sources, and computer asset data from inventory tools and internal databases.
2) **Graph construction.** Two subgraphs are built. In the Cyber Threat Graph, threat intelligence entities are represented as nodes and linked through typed relationships. In the Computer Asset Graph, Common Platform Enumeration (CPE) identifiers are linked to the corresponding vulnerabilities in the Cyber Threat Graph.
3) **Graph integration.** The two graphs are merged through entity alignment and entity resolution. The authors describe this step conceptually as an NLP-based process that uses named entity recognition (NER) and embeddings, with aligned node pairs represented as a product-combination graph. In practice, however, the integration they implemented relied on string- and text-similarity matching of entity names and descriptions between the CWE and CAPEC graphs.
4) **GNN enhancement.** The authors propose using graph neural networks (GNNs) to predict missing links, such as hidden attack paths or previously unidentified threat–asset relationships.
5) **Insight generation.** The authors propose using Neo4j for monitoring and visualization.

This work is closely related to our research, as it addresses many of the challenges central to our objectives. However, it concludes at the graph unification stage, and its link prediction, monitoring, and visualization components remain proposals rather than implementations. Our work extends this line of research by adopting their knowledge graph construction pipeline and building a context-aware vulnerability classification model on top of it.

Automated vulnerability classification has been widely studied to accelerate vulnerability triage and CVE-to-CWE mapping. Dong et al. In [3] fine-tuned BERT to jointly classify vulnerability categories and hazard levels from crawled vulnerability reports, achieving only 61.99–65.02% accuracy. These results highlight the difficulty of text-based classification when the label space becomes large and highly imbalanced. Das et al. In [8] addressed this limitation by formulating CVE-to-CWE assignment as a link-prediction problem rather than direct classification, achieving 89–98% accuracy on frequent CWE classes and 48–76% on rare classes under a temporal train/test split. Despite these improvements, both approaches rely primarily on vulnerability descriptions and the CWE hierarchy. Neither explicitly represents the asset affected by a vulnerability or the operational environment in which it is deployed.

Knowledge graph approaches have been explored to integrate and reason over cybersecurity knowledge from multiple sources. Rahman et al. In [4] developed an automated pipeline for periodically ingesting CVE, CWE, CAPEC, and ATT&CK data into a Neo4j graph containing more than 300,000 nodes, enabling multi-hop traversal and topology-based link scoring. Usama et al. In [9] constructed separate threat-centric and asset-centric graphs and integrated them through entity alignment, with the resulting graphs validated against external gold standards. Other studies have focused on constructing knowledge graphs directly from unstructured CTI reports. Piplai et al. In [13] used CRF-based entity extraction together with a UCO-based schema that includes relations such as *mitigates*, while [1] combined RoBERTa-BiLSTM-CRF extraction with progressive entity linking and RotatE-based novelty detection. Wang et al. In [6] applied topic clustering and BERT-based semantic role labeling to extract subject–action–object triples from open-source threat reports. Although these approaches demonstrate the effectiveness of automated entity and relation extraction, their resulting graphs primarily represent the broader threat landscape, including actors, malware, and techniques, rather than relationships that directly characterize the risk of a vulnerability to a specific asset.

Some studies have moved toward a more asset-centric representation. Zou et al. In [5] jointly modeled network topology, system configuration, and vulnerability information and used the resulting graph for threat-path prediction and probability-based alerting. However, the approach does not explicitly define a multi-level risk classification based on the environmental context of an individual vulnerability. Li et al. In [11] constructed a vulnerability ontology linking vulnerabilities with products, vendors, risk levels, and categories and supported natural-language queries over the graph. While useful for vulnerability knowledge retrieval, the ontology has limited contextual depth and does not reason over operational conditions or threat-intelligence information.

Beyond graph construction, other studies have investigated graph refinement and reasoning to improve the quality and analytical capability of cybersecurity knowledge graphs. Shi et al. In [7] showed that pruning disconnected entities and merging version-specific entities before graph embedding can improve link-prediction performance on a CPE–CVE–CWE graph. Andrejeus et al. In [12] proposed a staged repair framework combining normalization, Markov-based entity-type smoothing, constrained LLM repair, and Skew Zero Forcing, recovering 30.9% of otherwise discarded schema-invalid CTI triples. Belhadj et al. In [10] combined graph neural networks with large language models for real-time graph validation and anomaly detection, although the evaluation was limited to synthetic microservice data. These studies improve graph completeness, consistency, or reasoning capability, but do not specifically address the use of graph-derived operational context for vulnerability risk prioritization.

Amin et al. In [2] is among the closest examples of an integrated classification and reporting pipeline. Their approach

combines honeypot-collected logs with an XGBoost classifier for attack-type detection, a Neo4j knowledge graph linking detected attack types to plausible CVEs, and a LoRA-fine-tuned LLaMA model for automated CTI report generation. While this demonstrates the feasibility of combining machine learning, knowledge graphs, and automated reporting, the classification focuses on four coarse-grained attack categories rather than vulnerability risk. Furthermore, the graph does not explicitly model asset criticality, operational exposure, or a multi-level risk outcome.

Overall, vulnerability classification and CTI knowledge graph construction have largely been investigated as separate problems. Existing classification approaches primarily infer vulnerability categories from textual descriptions, whereas knowledge graph approaches focus on representing and connecting cybersecurity entities. Even integrated pipelines such as [2] rely on general-purpose graph relationships rather than a schema designed around an environment-specific risk question. Similarly, asset-oriented approaches [5], [11] provide contextual representations but do not explicitly couple that context with a multi-level vulnerability risk assessment. These limitations motivate a Vulnerability Context Knowledge Graph (VCKG) designed to determine how the risk posed by a vulnerability changes according to the characteristics and operational context of the affected asset.

III. Our Proposed System

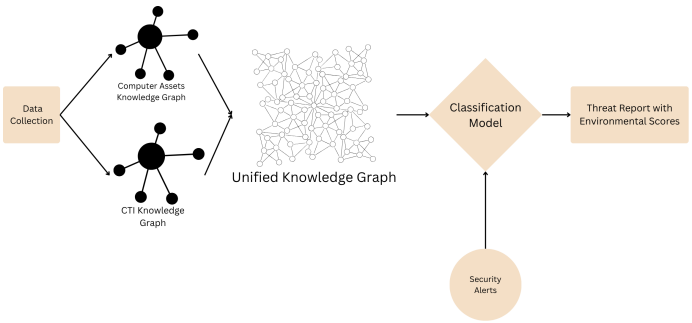


Fig. 1. Our proposed System Overview model

A. System Overview

Our system is designed to build on top of on the work of [1] to create a context-aware classification system using knowledge graph as its basis.

The proposed system consists of major parts as follows:

1) **Unified Knowledge Graph Construction.** We adopt the three-phase construction framework of Usama et al. [1] (data collection, knowledge graph construction, and graph integration through entity alignment and resolution) and adapt its integration phase to our setting. Whereas [1] aligned taxonomies through string similarity, our CTI sources share standardized identifiers, allowing deterministic alignment; the resolution effort is instead concentrated where [1] leaves the problem open, namely mapping an organization's deployed software to CPE identifiers.

- **Data Collection Phase** For the CTI knowledge graph, we use only structured authoritative CTI sources including:
  - **MITRE CWE** List of software and hardware weaknesses.
  - **MITRE CAPEC** List and Classification of attack patterns.
  - **MITRE ATT&CK** Knowledge base of attack tactics and techniques from real-world observations.
  - **CVE Program cvelist5** List and records of known vulnerabilities and exposures.
  - **NVD CVE yearly feeds** CVE feeds from the NVD database. This serves as a redundant data to fill in the information gaps of cvelist5.

  For Computer Assets graph, we use assets report from inventory scanner as the source. We aggregate the report to lists of assets and its attribute and details. We also includes Confidentiality, Integrity, Availability (C.I.A.) requirements to the list of assets attributes. This requirements will be used in the later stage of the framework.
- **Knowledge Graph Construction.** Each knowledge graph is defined as a labeled, directed multi-graph:

$$G = (V, E, \lambda, \rho) \tag{1}$$

where V is the set of nodes, $E \subseteq V \times V$ is the set of directed edges, $\lambda : V \to L$ is a function that assigns each node a type label from the set of node types L, and $\rho : E \to R$ is a function that assigns each edge a relation type from the set of relation types R.

We first construct the CTI knowledge graph, which comprises nine node types and eleven relation types. CWE weaknesses serve as the central hub of each threat subgraph. Each weakness is connected to CVE vulnerabilities (`EXPLOITED_BY`), CAPEC attack patterns (`CAUSES_WEAKNESS`), ATT&CK techniques and tactics (`USES_TECHNIQUE` and `IMPLEMENTS`), threat groups, mitigations (`MITIGATED_BY`), detection methods (`DETECTED_BY`), and affected platforms (`AFFECTS`). The hierarchical structure of the threat taxonomies is preserved through the `IS_PARENT` and `SUBTECHNIQUE_OF` relations. Finally, CPE nodes are linked to their corresponding CVE vulnerabilities through `VULNERABLE_TO` relations. The resulting graph schema is illustrated in Fig. 2.
- **Computer Assets Graphs Construction**
  The computer asset graph $G_A = (V_A, E_A)$ represents the environment that vulnerability intelligence is applied to: which hosts exist, where they sit

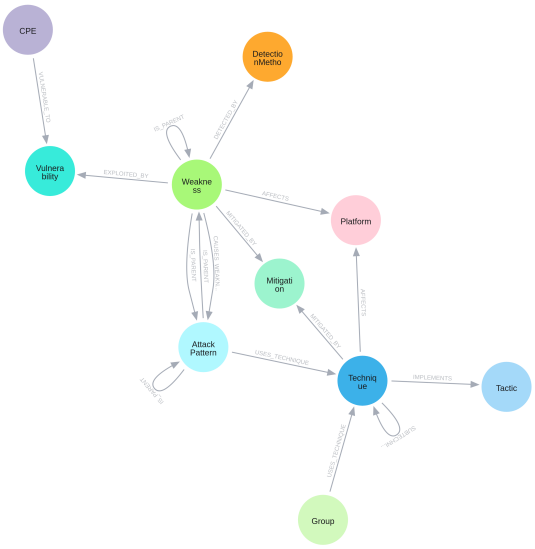


Fig. 2. CTI Knowledge Graph Schema

in the network, what software they run, how they communicate, and how important they are to the organization. Following the construction approach of Usama et al. [1], in which asset data are taken from the organization's own inventory, the graph is *domain-specific*: it describes one network and is rebuilt when that network changes. We build it for a laboratory network so that asset values and attack scenarios are controlled during evaluation. This section describes the data sources, the graph schema, and the construction procedure.

The computer asset knowledge graph is designed to represent the network infrastructure and its associated security context through five node types and six relation types. Asset nodes form the core of the graph, representing hosts in the monitored network and storing key attributes, including hostname, IP address, operating system, role, and the CVSS v3.1 security requirements CR, IR, and AR. Each asset is associated with a network zone through the `IN_ZONE` relation, while communication between assets is represented by `CONNECTS_TO`. The `ALLOWED_TO_REACH` relation captures inter-zone communication permitted by firewall rules. At the service and software levels, the `RUNS` relation links assets to the services they host, and `IMPLEMENTED_BY` associates each service with its corresponding software product. Software products are further mapped to standardized Common Platform Enumeration (CPE) identifiers through the `IDENTIFIED_AS` relation. These CPE nodes are shared with the CTI knowledge graph, enabling the identification of vulnerabilities affecting software deployed on each asset through the `VULNERABLE_TO` relation. This integration establishes a link between network assets, their software dependencies, and known vulnerabilities, providing the contextual foundation for subsequent risk assessment. The resulting graph schema is illustrated.

- **Graph Integration and Resolution** This is the final step of constructing the unified knowledge graph. In this step, we join the two KG through the shared common CPE nodes in the two graphs. Where the CPE nodes in CTI knowledge graph represents the platform CVE is found on and in CA knowledge graph it represents what platform the asset operates on. We link the two together with an identifier-based linking method.

$$G_{CTI} = (V_C, E_C, \lambda_C, \rho_C) \quad (2)$$

$$G_{CA} = (V_A, E_A, \lambda_A, \rho_A) \quad (3)$$

Where G_{CTI} and G_{CA} represent the CTI and CA knowledge graphs respectively. Given the knowledge graphs, an alignment is a set $M \subseteq V_C \times V_A$ of pairs referring to the same real-world entity. The unified graph is

$$G_{\text{VKG}} = ((V_C \cup V_A)/\sim_M,\ E_C \cup E_A,\ \lambda,\ \rho) \quad (4)$$

where $\sim_M$ is the equivalence relation induced by M and aligned nodes are merged into a single node that keeps the union of their edges.

References

[1] M. Usama, K. Nagaty and N. Elmaghawry, "Construction of a Unified Knowledge Graph for Cyber Threat Intelligence," 2025 International Conference on Computer and Applications (ICCA), Bahrain, Bahrain, 2025, pp. 1-9

[2] Yukun Dong, Yeer Tang, Xiaotong Cheng, Yufei Yang, DeKeDVer: A deep learning-based multi-type software vulnerability classification framework using vulnerability description and source code, Information and Software Technology, Volume 163, 2023, 107290, ISSN 0950-5849,

[3] X. Liu, Y. Tian, L. Tang and S. Zeng, "A Framework for Automated Cyber Threat Intelligence Extraction Based on Knowledge Graph," 2026 38th Chinese Control and Decision Conference (CCDC), Nanjing, China, 2026, pp. 4655-4660

[4] M. Rahman et al., "ATT&CK to CVE: A Large-Scale Automated Knowledge Graph for Threat Intelligence," 2025 IEEE International Conference on Data Mining Workshops (ICDMW), Washington, DC, USA, 2025, pp. 1044-1054

[5] A. Piplai, S. Mittal, A. Joshi, T. Finin, J. Holt and R. Zak, "Creating Cybersecurity Knowledge Graphs From Malware After Action Reports," in IEEE Access, vol. 8, pp. 211691-211703, 2020

[6] S. S. Das, E. Serra, M. Halappanavar, A. Pothen and E. Al-Shaer, "V2W-BERT: A Framework for Effective Hierarchical Multiclass Classification of Software Vulnerabilities," 2021 IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA), Porto, Portugal, 2021, pp. 1-12

[7] W. Wang, X. Han, Q. Zhao, L. Han, G. Wei and D. Xu, "Threat intelligence analysis based on multi-dimensional feature topic clustering and knowledge graph," 2023 5th International Conference on Frontiers Technology of Information and Computer (ICFTIC), Qiangdao, China, 2023, pp. 297-301

[8] C. Ma, D. Zhang, J. Zhang, L. Wang, H. Li and Z. Gu, "Cybersecurity Knowledge Graph Enables Targeted Data Collection for Cybersecurity Analysis," 2023 8th International Conference on Data Science in Cyberspace (DSC), Hefei, China, 2023, pp. 583-590

[9] Y. Li, Y. Guo, Y. Hao, Y. Wang, A. Yin and Y. Liu, "Intelligent Answer System Based on Vulnerability Knowledge Graph," 2021 7th International Conference on Computer and Communications (ICCC), Chengdu, China, 2021, pp. 1646-1651

[10] Y. Yang and Y. Zhao, "Network Security Threat Intelligence Modeling Based on Knowledge Graph," 2024 5th International Conference on Computer, Big Data and Artificial Intelligence (ICCBD+AI), Jingdezhen, China, 2024, pp. 154-158
[11] Z. Shi, N. Matyunin, K. Graffi and D. Starobinski, "Uncovering Product Vulnerabilities with Threat Knowledge Graphs," 2022 IEEE Secure Development Conference (SecDev), Atlanta, GA, USA, 2022, pp. 84-90
[12] TTMFN: Two-stream Transformer-based Multimodal Fusion Network for Survival Prediction, arXiv, 2023.
[13] Security model utilizing multi-channel data with risk-entity facing cybersecurity alert engine and portal (CVSS/NIST-style threshold definition).
[14] Castro-García, N., Muñoz Castañeda, Á. L., & Fernández-Rodríguez, M. (2020). Machine learning for automatic assignment of the severity of cybersecurity events. Computational and Mathematical Methods, 2, e1072.